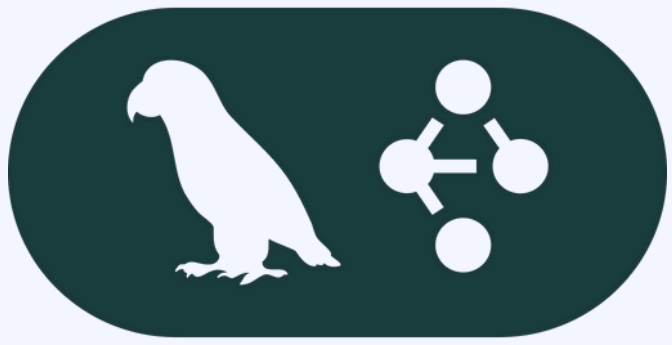
Mowka
Grow with Confidence



RAG with LangGraph

This Is How Production-Grade RAG Is Actually Built

Shubham Kansal
Founder | Mouka
https://mowka.in/

Understanding RAG

Core Concept

RAG combines document retrieval with AI generation for accurate answers

Step 1

Phase 1: Search and locate relevant information from your files

Step 2

Phase 2: Use retrieved content to craft precise responses

Why RAG? Provides up-to-date answers from your own data

Key benefit: Reduces hallucinations by grounding answers in real documents

Perfect for: Knowledge bases, documentation, internal Q&A systems

Shubham Kansal

Founder | https://mowka.in

◎ Step 1: Set Up Your Project Directory

- Organize your workspace with a logical folder layout
- Separate your data files from application code
- Follow single responsibility principle for each file

CODE

```
rag-langgraph/
 docs/
 index.py
 rag_graph.py
 run.py
```

Shubham Kansal

Founder | https://mowka.in

Step 2: Install Dependencies

- LangGraph → Manages the RAG pipeline execution
- Chroma → Stores vector representations of your documents
- LangChain → Handles file loading and text segmentation

CODE

```
 pip install -U langgraph langchain langchain-
community langchain-text-splitters chromadb
tiktoken
pip install -U langchain-openai
```

Shubham Kansal

Founder | https://mowka.in

◎ Step 3: Prepare Your Document Files

- Add text or markdown files to your /docs directory
- Every file will be indexed and made queryable
- Larger document sets provide more comprehensive responses

CODE

```
docs/
 faq.txt
 handbook.txt
 product_notes.md
```

Shubham Kansal

Founder | https://mowka.in

Step 4: Import Documents into Your Application

- Scan the /docs folder and read all available files
- Transform each file into LangChain Document instances
- Prepare documents for text segmentation

CODE

```
 # index.py
from pathlib import Path
from langchain_community.document_loaders import
TextLoader

DOCS_DIR = Path("docs")

def load_docs():
    docs = []
    for fp in DOCS_DIR.glob("*"):
        if fp.is_file():
            docs.extend(TextLoader(str(fp),
encoding="utf-8").load())
    return docs
```

Step 5: Segment Documents into Smaller Units

- Divide lengthy documents into manageable text segments
- Use 800 character chunks with 120 character overlap between segments
- Overlapping sections preserve context across boundaries

CODE

```
 # index.py
from langchain_text_splitters import
RecursiveCharacterTextSplitter


splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120
)
chunks = splitter.split_documents(load_docs())
```

Shubham Kansal

Founder | https://mowka.in

Step 6: Generate Embeddings and Index in Vector Database

- Transform text segments into vector representations
- Save embeddings in Chroma for efficient semantic search
- Execute once: python index.py (needs OPENAI_API_KEY environment variable)

CODE

```
 # index.py
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import
Chroma

embeddings = OpenAIEmbeddings()

Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="chroma_db",
    collection_name="rag_docs",
)
print("✅ Index built")
```

Step 7: Design the LangGraph State Schema

- State represents the data structure passed between graph nodes
- Includes three fields: user question, retrieved documents, and final answer
- TypedDict provides compile-time type checking

CODE

```
 # rag_graph.py
from typing import TypedDict, List
from langchain_core.documents import Document


class RAGState(TypedDict):
    question: str
    docs: List[Document]
    answer: str
```


Shubham Kansal

Founder | https://mowka.in

◎ Step 8: Implement the Retrieval Node

- Passes retrieved chunks forward in the state object

CODE

```
# rag_graph.py
from langchain_community.vectorstores import
Chroma
from langchain_openai import OpenAIEmbeddings

vectorstore = Chroma(
    persist_directory="chroma_db",
    collection_name="rag_docs",
    embedding_function=OpenAIEmbeddings(),
)
retriever =
vectorstore.as_retriever(search_kwargs={"k": 4})

def retrieve(state: RAGState) → RAGState:
    q = state["question"]
    docs = retriever.invoke(q)
    return {"question": q, "docs": docs, "answer":
""}
```

Step 9: Configure LLM and Format Context

- Set up ChatOpenAI instance using the gpt-4o-mini model
- Pull the user question and retrieved documents from state
- Merge all document segments into a single context string

CODE

```
 # rag_graph.py
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage,
HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini",
temperature=0)

def generate(state: RAGState) → RAGState:
    q = state["question"]
    docs = state["docs"]

    # Combine all document chunks into context
    context = "\n\n".join([d.page_content for d in
docs])
```

◎ Step 10: Create the Answer Generation Function

- Construct a prompt with system instructions to use only provided context
- Insert the context and user question into a HumanMessage
- Invoke the language model and store the response in state

CODE

```
 # rag_graph.py (continued)
    messages = [
        SystemMessage(content=(
            "Answer using ONLY the context. "
            "If missing, say you don't have that
info in the documents."
        )),
        HumanMessage(content=f"Question:
{q}\n\nContext:\n{context}")
    ]

    resp = llm.invoke(messages)
    return {**state, "answer": resp.content}
```

Step 11: Assemble the LangGraph Pipeline

- LangGraph automatically manages the workflow execution
- Extensible design allows adding additional nodes (filtering, reranking, etc.)

CODE

```
 # rag_graph.py
from langgraph.graph import StateGraph, START, END

def build_graph():
    g = StateGraph(RAGState)
    g.add_node("retrieve", retrieve)
    g.add_node("generate", generate)

    g.add_edge(START, "retrieve")
    g.add_edge("retrieve", "generate")
    g.add_edge("generate", END)

    return g.compile()

rag_app = build_graph()
```

Step 12: Launch and Test Your RAG Application

- Interactive command-line interface: input questions and receive answers
- Display information about which document segments were utilized
- Your RAG-powered assistant is now operational! 🎉

CODE

```
 # run.py
from rag_graph import rag_app

while True:
    q = input("\nAsk: ").strip()
    if not q:
        break

    result = rag_app.invoke({"question": q,
"docs": [], "answer": ""})
    print("\nAnswer:\n", result["answer"])
    print("\nChunks used:", len(result["docs"]))
```

Shubham Kansal

Founder | Mouka

mowka.in

Follow on LinkedIn

Repost This Carousel

Ready to Find Your Perfect Engineering Match?

Let's Connect & Chat →